

Qt Base Course

Day 2

Advanced GUI Techniques

Day 2 — Agenda

Drag & Drop and Clipboard

- Drag sources and drop targets
- MIME types — `QMimeData`
- Inter-process drag and drop
- Clipboard: `QClipboard`

Item-Based Widgets

- `QListWidget`, `QTableWidget`, `QTreeWidget`
- Item roles, icons, check states, sorting

Model-View Widgets

- Model-View-Delegate architecture
- `QAbstractItemModel` and subclasses
- Views: `QListView`, `QTableView`, `QTreeView`
- `QSortFilterProxyModel`

2D Graphics — View, Scene, Item

- Graphics View Framework
- `QGraphicsScene` and `QGraphicsView`
- Built-in and custom `QGraphicsItem`
- Transformations, z-ordering, collision detection

Part 1 — Drag & Drop and Clipboard

Enabling Drag — The Source Side

```
class DragLabel : public QLabel {
protected:
    void mousePressEvent(QMouseEvent *event) override {
        if (event->button() != Qt::LeftButton) return;
        m_dragStart = event->pos();
    }

    void mouseMoveEvent(QMouseEvent *event) override {
        if (!(event->buttons() & Qt::LeftButton)) return;
        if ((event->pos() - m_dragStart).manhattanLength()
            < QApplication::startDragDistance()) return;

        auto *drag = new QDrag(this);
        auto *mime = new QMimeData;
        mime->setText(text());
        drag->setMimeData(mime);
        drag->setPixmap(grab()); // visual drag thumbnail
        drag->exec(Qt::CopyAction | Qt::MoveAction);
    }
private:
    QPoint m_dragStart;
};
```

Enabling Drop — The Target Side

```
class DropZone : public QWidget {
public:
    explicit DropZone(QWidget *parent = nullptr) : QWidget(parent) {
        setAcceptDrops(true);    // ← must opt in
    }

protected:
    void dragEnterEvent(QDragEnterEvent *event) override {
        if (event->mimeTypeData()->hasText())
            event->acceptProposedAction();
    }

    void dragMoveEvent(QDragMoveEvent *event) override {
        event->acceptProposedAction();
    }

    void dropEvent(QDropEvent *event) override {
        label->setText(event->mimeTypeData()->text());
        event->acceptProposedAction();
    }
};
```

MIME Types in Qt

`QMimeType` can carry multiple formats simultaneously:

```
auto *mime = new QMimeType;

mime->setText("plain text");           // text/plain
mime->setHtml("<b>rich text</b>");       // text/html
mime->setUrls({ QUrl("file:///home/user/doc.pdf") });
mime->setImageData(QImage(":/photo.png"));

// Custom binary format
mime->setData("application/x-myapp-item",
            serialiseItem(myItem));
```

Check availability before reading:

```
if (event->mimeType()->hasUrls()) {
    for (const QUrl &url : event->mimeType()->urls())
        openFile(url.toLocalFile());
}
```

Inter-Process Drag and Drop

Qt handles inter-process drops automatically — the OS mediates via native MIME:

- Drag a file from Windows Explorer / macOS Finder → `hasUrls()` is true in your app
- Drag text from a browser → `hasText()` / `hasHtml()` is true
- Your app can advertise custom MIME types understood by other Qt apps

No extra code needed — `QMimeData` serialises across process boundaries via the OS clipboard protocol.

Clipboard

```
QClipboard *clipboard = QApplication::clipboard();

// Write
clipboard->setText("Hello from Qt");
clipboard->setPixmap(QPixmap(":/screenshot.png"));

// Read
if (!clipboard->text().isEmpty())
    label->setText(clipboard->text());

// React to changes
connect(clipboard, &QClipboard::dataChanged, this, [this]() {
    pasteAction->setEnabled(!QApplication::clipboard()->text().isEmpty());
});
```

On Linux, connect to `QClipboard::selectionChanged` too — X11 has a separate primary selection.

Part 2 — Item-Based Widgets

QListWidget

```
auto *list = new QListWidget(this);

// Add items
list->addItem("Apple");
list->addItem("Banana");

auto *item = new QListWidgetItem(QIcon(":/fruit.png"), "Cherry");
item->setCheckState(Qt::Unchecked);
item->setData(Qt::UserRole, 42);           // store arbitrary payload
list->addItem(item);

// React to selection
connect(list, &QListWidget::currentItemChanged,
        this, [](QListWidgetItem *current, QListWidgetItem *) {
            qDebug() << "Selected:" << current->text();
        });
```

QTableWidget

```
auto *table = new QTableWidget(3, 2, this);    // 3 rows, 2 columns
table->setHorizontalHeaderLabels({"Name", "Score"});

table->setItem(0, 0, new QTableWidgetItem("Alice"));
table->setItem(0, 1, new QTableWidgetItem("95"));
table->setItem(1, 0, new QTableWidgetItem("Bob"));
table->setItem(1, 1, new QTableWidgetItem("87"));

// Sort by column 1 ascending
table->sortItems(1, Qt::AscendingOrder);

// Resize columns to content
table->resizeColumnsToContents();
```

QTreeWidget

```
auto *tree = new QTreeWidget(this);
tree->setHeaderLabels({"Item", "Value"});

// Top-level item
auto *root = new QTreeWidgetItem(tree, {"Root", ""});

// Children
auto *child1 = new QTreeWidgetItem(root, {"Child A", "10"});
auto *child2 = new QTreeWidgetItem(root, {"Child B", "20"});
Q_UNUSED(child1); Q_UNUSED(child2);

root->setExpanded(true);

// Iterate all items
QTreeWidgetItemIterator it(tree);
while (*it) {
    qDebug() << (*it)->text(0);
    ++it;
}
```

Item Roles

Qt items carry multiple data roles — use them to separate concerns:

Role	Purpose
<code>Qt::DisplayRole</code>	Text shown to the user
<code>Qt::DecorationRole</code>	Icon (<code>QIcon</code> or <code>QPixmap</code>)
<code>Qt::EditRole</code>	Value used when editing
<code>Qt::CheckStateRole</code>	<code>Qt::Checked</code> / <code>Qt::Unchecked</code>
<code>Qt::ToolTipRole</code>	Tooltip string
<code>Qt::UserRole</code>	Your own data (any <code>QVariant</code>)
<code>Qt::UserRole + 1</code> etc.	Additional custom roles

Part 3 — Model-View Widgets

Why Model-View?

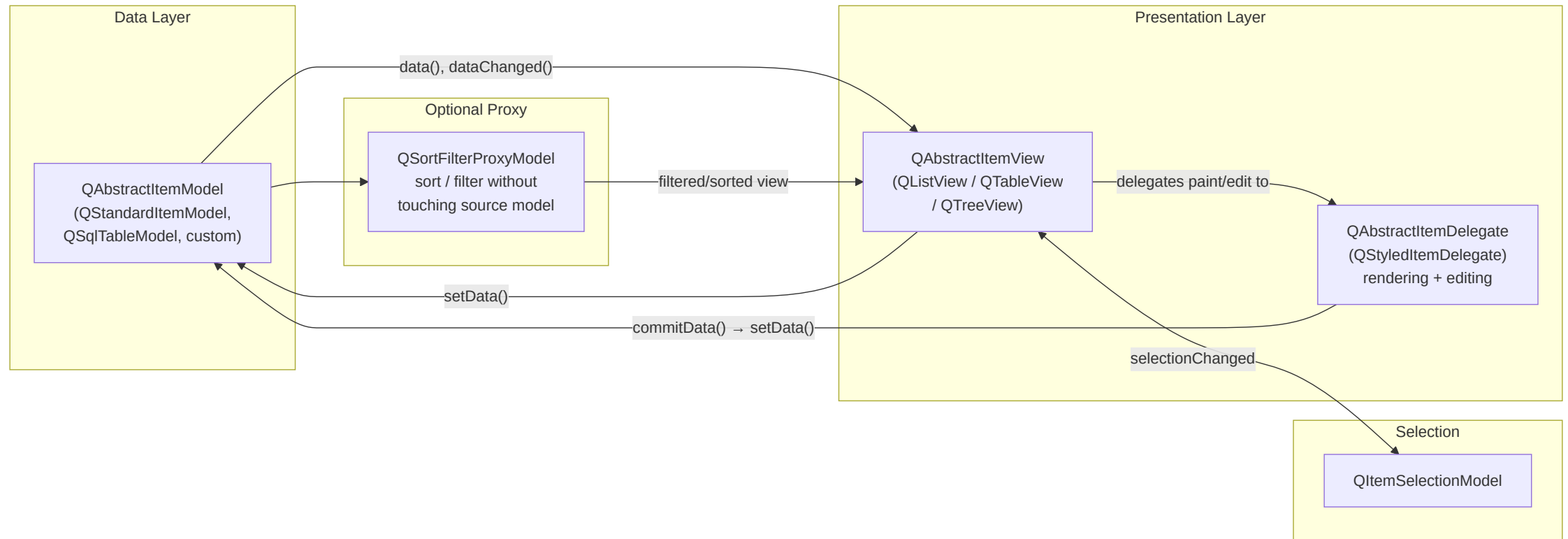
Item-based widgets (`QListWidget` , `QTableWidget`) mix data and presentation — fine for small, static data.

Model-View separates them:

Concern	Class
Data	<code>QAbstractItemModel</code> subclass
Display	<code>QListView</code> , <code>QTableView</code> , <code>QTreeView</code>
Editing	<code>QAbstractItemDelegate</code> subclass
Selection	<code>QItemSelectionModel</code>

Benefits: same model → multiple views, proxy filtering, SQL/DB binding, unit-testable data layer.

Model-View Architecture



Implementing a Custom List Model

```
class FruitModel : public QAbstractListModel {
    Q_OBJECT
public:
    int rowCount(const QModelIndex &parent = {}) const override {
        return parent.isValid() ? 0 : m_data.size();
    }

    QVariant data(const QModelIndex &index, int role = Qt::DisplayRole) const override {
        if (!index.isValid() || index.row() >= m_data.size())
            return {};
        if (role == Qt::DisplayRole)
            return m_data.at(index.row());
        return {};
    }

    void append(const QString &item) {
        beginInsertRows({}, m_data.size(), m_data.size());
        m_data.append(item);
        endInsertRows();    // ← notifies views automatically
    }

private:
    QStringList m_data;
};
```

Connecting Model to a View

```
auto *model = new FruitModel(this);
model->append("Apple");
model->append("Banana");
model->append("Cherry");

auto *view = new QListView(this);
view->setModel(model);           // view asks model for data

// React to selection
auto *sel = view->selectionModel();
connect(sel, &QItemSelectionModel::currentChanged,
        this, [model](const QModelIndex &idx) {
            qDebug() << model->data(idx).toString();
        });
```

QStandardItemModel — General-Purpose Tree

```
auto *model = new QStandardItemModel(this);
model->setHorizontalHeaderLabels({"Name", "Size"});

auto *root = model->invisibleRootItem();

auto *dir  = new QStandardItem(QIcon(":/folder.png"), "Documents");
auto *file = new QStandardItem(QIcon(":/file.png"),    "report.pdf");
auto *size = new QStandardItem("2.4 MB");
dir->appendRow({ file, size });
root->appendRow(dir);

auto *treeView = new QTreeView(this);
treeView->setModel(model);
treeView->expandAll();
```

QSortFilterProxyModel

```
auto *sourceModel = new FruitModel(this);
sourceModel->append("Apple");
sourceModel->append("Apricot");
sourceModel->append("Banana");

auto *proxy = new QSortFilterProxyModel(this);
proxy->setSourceModel(sourceModel);
proxy->setFilterCaseSensitivity(Qt::CaseInsensitive);
proxy->setSortCaseSensitivity(Qt::CaseInsensitive);

auto *view = new QListView(this);
view->setModel(proxy);    // view sees the proxy – source untouched

// Live filter from a search box
connect(searchEdit, &QLineEdit::textChanged,
        proxy, &QSortFilterProxyModel::setFilterFixedString);
```

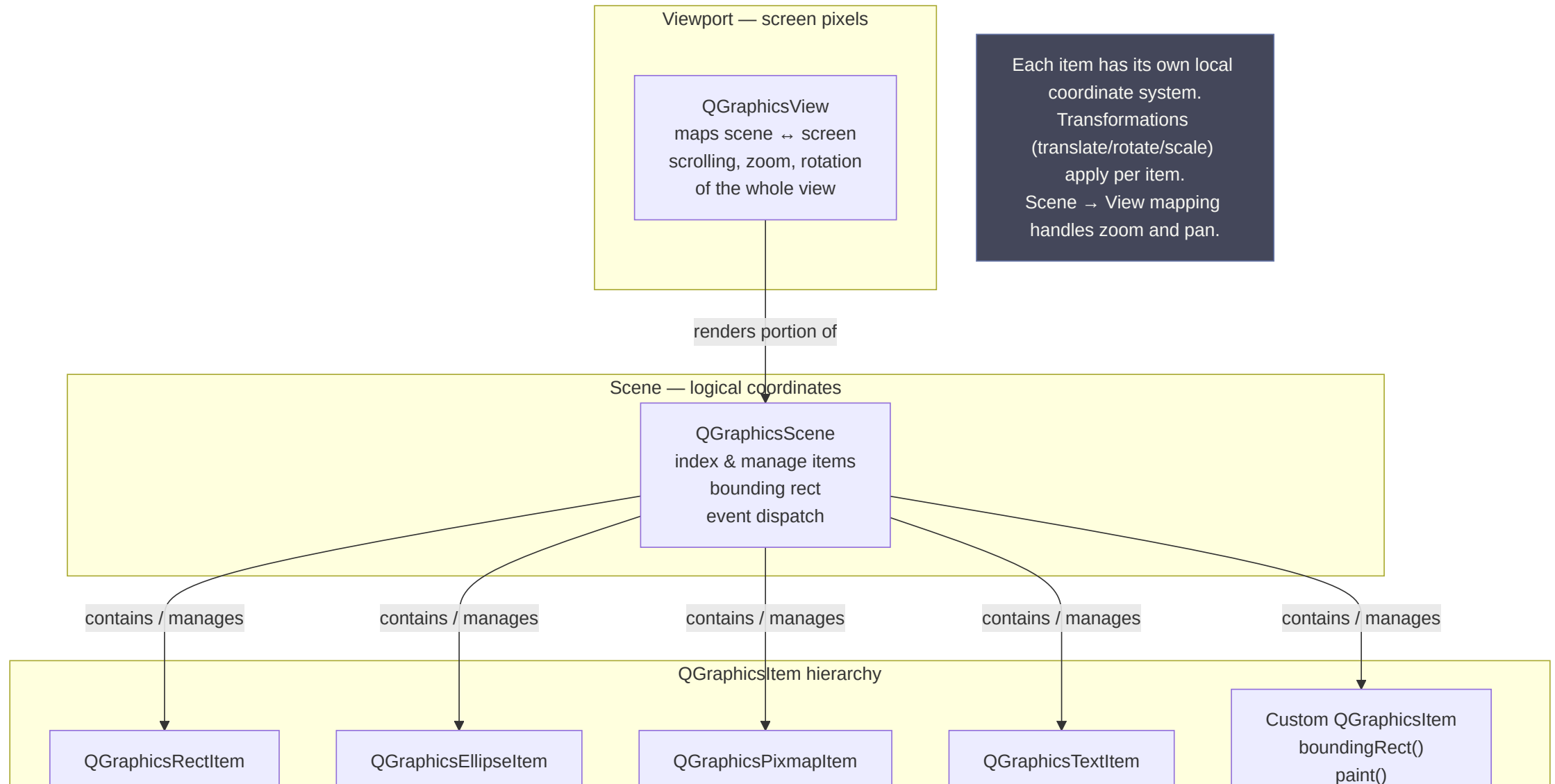
Custom Delegate

```
class StarDelegate : public QStyledItemDelegate {
public:
    void paint(QPainter *painter,
               const QStyleOptionViewItem &option,
               const QModelIndex &index) const override {
        int stars = index.data(Qt::UserRole).toInt();
        // draw filled stars based on rating
        for (int i = 0; i < 5; ++i)
            painter->drawText(option.rect.adjusted(i * 16, 0, 0, 0),
                              i < stars ? "★" : "☆");
    }

    QWidget *createEditor(QWidget *parent,
                           const QStyleOptionViewItem &,
                           const QModelIndex &) const override {
        auto *spin = new QSpinBox(parent);
        spin->setRange(0, 5);
        return spin;
    }
};
```

Part 4 — 2D Graphics: View, Scene, Item

Graphics View Framework



Coordinate Systems

System	Unit	Who uses it
Item coordinates	Logical, local to item	<code>paint()</code> , <code>boundingRect()</code>
Scene coordinates	Logical, global canvas	<code>QGraphicsScene::itemAt()</code> , <code>addItem()</code>
View coordinates	Pixels on screen	Mouse events, viewport geometry

```
// Convert between coordinate systems
QPointF scenePos = view->mapToScene(viewPoint);
QPointF itemPos  = item->mapFromScene(scenePos);
```

Setting Up a Scene and View

```
auto *scene = new QGraphicsScene(this);
scene->setSceneRect(-200, -200, 400, 400);

// Add built-in items
scene->addRect(-50, -50, 100, 100,
              QPen(Qt::black, 2), QBrush(QColor(80, 250, 123))));
scene->addEllipse(-30, -30, 60, 60,
                  QPen(Qt::red, 2));
auto *text = scene->addText("Qt Graphics View");
text->setPos(-60, 70);

auto *view = new QGraphicsView(scene, this);
view->setRenderHint(QPainter::Antialiasing);
view->setDragMode(QGraphicsView::ScrollHandDrag);
```

Custom QGraphicsItem

```
class ArrowItem : public QGraphicsItem {
public:
    QRectF boundingRect() const override {
        return QRectF(-5, -50, 10, 100);    // bounding box in local coords
    }

    void paint(QPainter *painter,
               const QStyleOptionGraphicsItem *,
               QWidget *) override {
        painter->setPen(QPen(Qt::black, 2));
        painter->drawLine(0, -50, 0, 50);
        // arrowhead
        painter->setBrush(Qt::black);
        painter->drawPolygon(QPolygonF({{0, -50}, {-5, -38}, {5, -38}}));
    }

    QPainterPath shape() const override {
        QPainterPath path;
        path.addRect(-5, -50, 10, 100);    // used for hit-testing
        return path;
    }
};
```

Item Transformations and Z-Ordering

```
QGraphicsItem *rect = scene->addRect(0, 0, 80, 50);

// Position
rect->setPos(100, 100);

// Transform
rect->setRotation(45);           // degrees
rect->setScale(1.5);
rect->setTransformOriginPoint(40, 25); // rotate around centre

// Z-ordering – higher value = drawn on top
rect->setZValue(2.0);

// Opacity
rect->setOpacity(0.7);

// Flags
rect->setFlag(QGraphicsItem::ItemIsMovable);
rect->setFlag(QGraphicsItem::ItemIsSelectable);
```

Collision Detection

```
// Check if two items overlap
if (playerItem->collidesWithItem(wallItem))
    handleCollision();

// Find all items the player collides with
const auto colliding = scene->collidingItems(playerItem);
for (auto *item : colliding) {
    if (auto *enemy = qgraphicsitem_cast<EnemyItem *>(item))
        enemy->destroy();
}

// Check collision with a path (e.g., laser beam)
QPainterPath beam;
beam.moveTo(laserPos);
beam.lineTo(laserEnd);

const auto hit = scene->items(beam);
```